Core 90 - Stacks and queues

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Stacks and queues | 3 questions | tap a title to jump to it

M	1. Decode String	1Point3Acres	2026-07-29
M	2. Expand Recursive Key-Value Placeholders in a Template	PracHub	2026-08-04
M	3. Visible People in a Line (Last Person, Then Everyone)	1Point3Acres	2026-10-07

STUDY PATH: question 76 of 167 (Core 90)

Priority score 6.81 | Stacks and queues | asked 83 days ago | frequency Low

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 1 of 3

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 57.7** | **30 DAYS 0** | **PRIORITY 45/228**

ABOUT THIS QUESTION

Topic: Core 90 - Stacks and queues

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

BRUTE FORCE

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decode String
#
# Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" ->
# "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).
#
# BRUTE FORCE (say it first): Recursive descent over the
# string
# A function that reads letters until it meets a digit
# (then reads the number, the bracketed substring
# recursively, and repeats it) or a closing bracket. It
# mirrors the grammar and is the oracle; the recursion
# depth equals the nesting depth. Time O(output length).
# Space O(depth).


# BRUTE FORCE (say it first): Recursive descent over the
# string | time O(output length)
def decode_recursive(s):
    pos = 0

    def parse():
        nonlocal pos
        out = []
        while pos < len(s) and s[pos] != "]":
            if s[pos].isdigit():
                num = 0
                while s[pos].isdigit():
                    num = num * 10 + int(s[pos])
                    pos += 1
                pos += 1  # skip '['
                inner = parse()
                pos += 1  # skip ']'
                out.append(inner * num)
            else:
                out.append(s[pos])
                pos += 1
        return "".join(out)

    return parse()


# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
# Answer: Compute its length (or a given character) from
# the stack without building it: keep (length so far,
# repeat count) frames and combine as prev + inner *
# times. The length can be computed for strings that could
# never fit in memory.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
# Answer: Walk the structure with lengths: at each repeat
# reduce k modulo the inner length to find where in the
# inner string it falls (LeetCode 880 style).
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
```

```
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None


if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

ALSO REPORTED · 1 more

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY PATH: question 77 of 167 (Core 90)

Priority score 5.81 | Stacks and queues | asked 77 days ago

MEDIUM | PracHub | Technical Screen | 2 of 3

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 57.7** | **30 DAYS 0** | **PRIORITY 46/228**

ABOUT THIS QUESTION

Topic: Core 90 - Stacks and queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

BRUTE FORCE TO SAY OUT LOUD

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

OPTIMAL IDEA

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

LIKELY FOLLOW-UPS

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

PITFALLS

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template, recursively
# resolving placeholders inside mapped values. Use
# memoized dependency traversal to preserve literal text,
# avoid repeated work, and explain cycle, unknown-key, and
# escaping policies.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.
#
# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing
# Replace every %KEY% found in the text by its value and
# repeat until nothing changes. Simple, but it never stops
# on a cycle (so it needs an iteration cap) and re-expands
# the same key every time it appears. Time O(depth *
# length). Space O(length).


import re


# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing | time O(depth * length)
_PH = re.compile(r"%([A-Za-z_][A-Za-z0-9_]*)%")
```

```
# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")
```

```
def expand_by_rewriting(
    template, mapping, max_rounds=50
):
    text = template
    for _ in range(max_rounds):
        new = _PH.sub(
            lambda m: mapping.get(
                m.group(1), m.group(0)
            ),
            text,
        )
        if new == text:
            return text
        text = new
    raise ValueError("cycle")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# FOLLOW-UP 1: A literal percent sign must be possible.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
```

```
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]
```

```
    return expand_text(template)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
```

```
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

ALSO REPORTED · 1 more

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY PATH: question 78 of 167 (Core 90)

Priority score 5.5 | Stacks and queues | asked 13 days ago | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 3 of 3

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 181/228

ABOUT THIS QUESTION

Topic: Core 90 - Stacks and queues

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

APPROACH

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what blocks the view (someone strictly taller than both), how ties are treated, which directions count in Part 2, and whether the code runs (it does not: trace by hand).

BRUTE FORCE TO SAY OUT LOUD

Check every pair against the people between them. O(n^2) for the last person, O(n^3) for everyone.

OPTIMAL IDEA

Part 1: walk from the person before the last towards the front, keeping the maximum height between the last person and the current one; the current person is visible when that maximum is at most the larger of the two heights. O(n). Part 2: for each person, everyone up to the first strictly taller person on the right is visible, then only people at least as tall as everyone from there on; a monotonic stack gives the next at-least-as-tall and next strictly-taller indexes and the chain length in O(n). Run it on the reversed line for the left side and add.

DRY-RUN THESE CASES

A single person, all equal heights, strictly increasing and decreasing lines, the example from the statement (the last person sees 6), a tie case you invent.

LIKELY FOLLOW-UPS

Only looking in one direction, blocking on equal heights, or returning who they see instead of how many.

PITFALLS

Using a strict comparison where the rule says taller than both. Forgetting that the first taller person is itself visible. Rereading the example without tracing it by hand.

HOW TO THINK ABOUT IT

Person A sees person B when nobody strictly between them is taller than both, which means the tallest person between them is at most as tall as the taller of A and B. Walking away from the last person and keeping the running maximum of everyone passed lets you test each person in O(1). For everyone at once the visible people to the right form a chain: everything up to the first taller person, then only the people who are at least as tall as everyone before them. A monotonic stack builds those chains for all positions in O(n).

Basis: LeetCode 1944, Number of Visible People in a Queue, with the rule that a person is hidden only by someone taller than both.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	**O(1)**

BRUTE FORCE

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).
#
# BRUTE FORCE (say it first): Test every pair
# For a pair (i, j) look at everyone between them and
# check that none is taller than both. Counting over all
# pairs is O(n^3). Fine as the oracle and for hand-tracing
# the example. Time O(n^3) for everyone, O(n^2) for the
# last person. Space O(1).


# BRUTE FORCE (say it first, it also checks the answers)
def sees(a, i, j):
    # the oracle: nobody strictly between is taller than
    # both
    lo, hi = min(i, j), max(i, j)
    return all(a[k] <= max(a[i], a[j])
               for k in range(lo + 1, hi))


# BRUTE FORCE (say it first, it also checks the answers)
def counts_brute(a):
    return [sum(1 for j in range(len(a))
                if j != i and sees(a, i, j))
            for i in range(len(a))]


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_right(a):
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
```

```
    least as tall as everyone from that person on."""
    n = len(a)
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]


if __name__ == "__main__":
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
```

```
    assert last_sees(line) == 6
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
    assert see_all([5, 5, 5]) == [2, 2, 2]
    import random
    for _ in range(300):
        a = [random.randint(1, 6)
             for _ in range(random.randint(0, 9))]
        assert see_all(a) == counts_brute(a)
        if a:
            assert last_sees(a) == counts_brute(a)[-1]
```

NOTES FROM CANDIDATES

Pitfalls: The code is written in a plain text editor that does not compile or run; trace the example by hand before declaring the solution done.
Confirm the direction of "see" for Part 2 (toward the front only, or both directions) and how equal heights are treated before coding; the example does not exercise ties.

Follow-ups: Generalize from the last person to every person in the line (Part 2).

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.